P.I. Redes / Oper

Proyecto: **Etapa #2**

Equipo: Los Padawans

Juan González Márquez - C4F588
Jeferson Marín Porras - C24549
Ariadna Ardila Gutiérrez - C4C646
Juan Sebastán Loazia Ospina B74200

Conexión

Protocolo: Intermediario - Intermediario

Los intermediarios se conectan mediante un switch centralizado usando IPv4

- El proceso inicia con un mensaje JOIN enviado por UDP
- El switch hace broadcast a los demás intermediarios registrados
- Los intermediarios responden con un HANDSHAKE para establecer conexión TCP/IP directa

Se utilizan puertos distintos a los clientes para diferenciar tipos de comunicación.
Cada conexión se maneja en un hilo independiente

Comunicación

Protocolo: Intermediario - Intermediario

Intercambio de paquetes:

- INTERMEDIARY_REQUEST (solicitudes)
- INTERMEDIARY_RESPONSE (respuestas)

- En el HANDSHAKE se intercambian tablas de figuras
- Cada intermediario mantiene un vector de sockets para comunicarse con otros
- Si no encuentra una figura localmente:
  - Consulta otros intermediarios

Cada hilo:

- Lee y escribe en su canal
- Procesa solicitudes o respuestas

El intermediario identifica al cliente original mediante la dirección incluida en los paquetes

Tabla de figuras

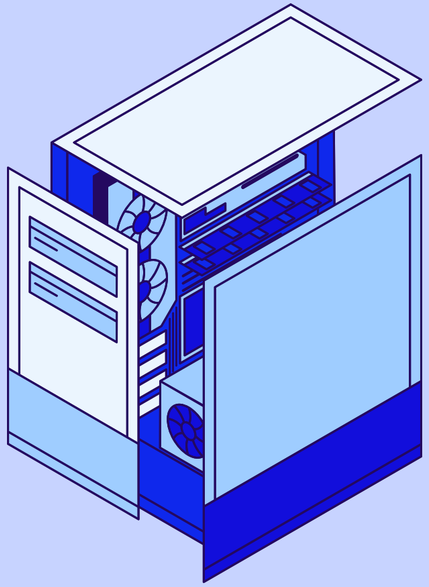
Protocolo: Intermediario - Intermediario

¿Qué es?

Consiste en una tabla global con figuras conocidas en la red

Permite:

- Validación rápida
- Evitar búsquedas innecesarias
- Coordinación entre intermediarios



Flujo de uso:

- Se consulta la tabla global
- Si no existe → FIGURE_NOT_FOUND
- Si existe → busca localmente
- Si no está → reenvío a otros intermediarios (multicast)
- Se usa la primera respuesta recibida

Bitácoras (**Logs**)

Estructura y propósito

Los logs registran el comportamiento del sistema en texto plano. Permiten:

- Trazabilidad
- Monitoreo
- Depuración del sistema

Formato:

[ENTIDAD] [FECHA_HORA] [TIPO_LOG] [MENSAJE]

Tipos de logs:

- SOLICITUD
- ENRUTAMIENTO
- RESPUESTA
- ERROR

Bitácoras (Logs)

Tipos de eventos registrados

Solicitudes:

- Inicio de operaciones (LIST_FIGURES, GET_FIGURE, ASK_FIGURE)

Enrutamiento:

- Decisiones del intermediario
- Consulta de tabla de rutas
- Reenvío entre intermediarios

Respuestas:

- Resultados del sistema (FIGURES_LIST, RETURN_FIGURE, NOT_FOUND)

Errores:

- Fallos de conexión
- Timeouts
- Servidores no disponibles

File System

Nuestro sistema de archivos es una implementación simplificada de un filesystem tipo Unix, basado en el uso de inodos y administración de bloques sobre un disco virtual representado por un archivo binario.

El sistema utiliza un archivo llamado “lego_data.bin” como disco, el cual está organizado en bloques de tamaño fijo de 512 bytes. Todas las operaciones de almacenamiento, lectura y escritura se realizan a nivel de bloques, simulando el comportamiento de un sistema de archivos real.

La arquitectura del filesystem está compuesta por las siguientes estructuras principales:

Superbloque

Contiene la información global del sistema, incluyendo el número total de bloques, bloques libres, y la ubicación del bitmap, la tabla de inodos y la zona de datos.

Bitmap de bloques

Se utiliza para gestionar la ocupación del disco. Cada posición del bitmap representa un bloque del disco, donde 0 indica que el bloque está libre y 1 que está ocupado.

Tabla de inodos

Cada archivo es representado por un inodo, el cual almacena metadatos como tamaño del archivo, fechas de creación y modificación, y punteros a los bloques donde se encuentran los datos. El sistema implementa:

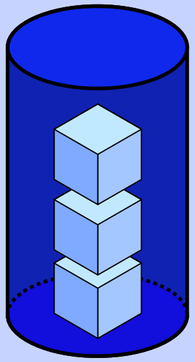
Punteros directos para archivos pequeños

Un puntero indirecto simple

Un puntero doble indirecto

Esto permite escalar el tamaño de los archivos más allá de los bloques directos.

File System



Zona de datos

Es la región del disco donde se almacenan los contenidos reales de los archivos y estructuras como el directorio raíz.

Directorios

El sistema implementa un directorio raíz simple, que almacena entradas de tipo nombre -> inodo. Cada entrada contiene el nombre del archivo y el índice del inodo correspondiente.

Operaciones Implementadas

- Creación del sistema de archivos (mkfs)
- Montaje (mount)
- Creación de archivos
- Escritura de datos
- Lectura de archivos
- Eliminación de archivos
- Listado del directorio

Durante la creación mkfs, se inicializan todas las estructuras, incluyendo el superbloque, el bitmap, la tabla de inodos y el directorio raíz. En el montaje, estas estructuras se cargan desde el disco a memoria.

Este diseño se inspira en sistemas de archivos reales tipo Unix como ext2, pero simplificado para fines del curso, permitiendo comprender conceptos fundamentales como la gestión de bloques, la organización mediante inodos y la persistencia en disco.

Simulación

Cliente

Único punto de interacción con el usuario. Pide al intermediario la lista de figuras LEGOS, o las partes que contiene cualquiera de las figuras listadas.

Death Star II

Intermediario

Recibe e interpreta las peticiones del cliente para el servidor. Es un punto medio de control entre le cliente y el servidor; maneja un cache que mantiene actualizada la lista de servidores disponibles, así como las figuras que contienen.

Legos disponibles

Servidores disponibles

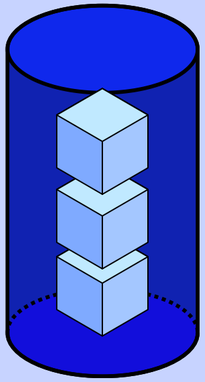
Intermediario2

Existe como una segunda forma de verificación para figuras que no esten almacenadas en el servidor oficial. De no ser encontradas se puede hacer un request externo al Intermediario2 para verificar disponibilidad.

Servidor

Centro de almacenamiento de datos donde se graba en memoria toda la informacion existente respecto a las figuras, sus segmentos, partes y metadata. Es el único que tiene acceso al filesystem y se encarga de procesar y resolver las peticiones que recibe a travez del intermediario.

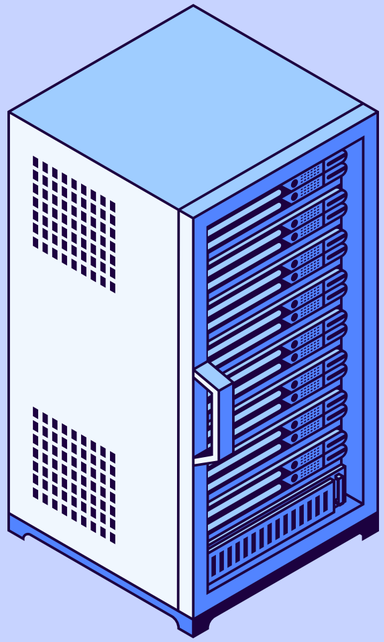
ServerMain



SERVIDOR

- Se crea un hilo por cada cliente
- Uso de std::thread para concurrencia
- Cada hilo ejecuta handleClient()

os.ecci.ucr.ac.cr



- Se recibe la solicitud HTTP (readHttpRequest)
- Se procesa la ruta (processRequest)
- Se genera la respuesta
- Se envía al cliente (Write)
- Se cierra la conexión (Close)

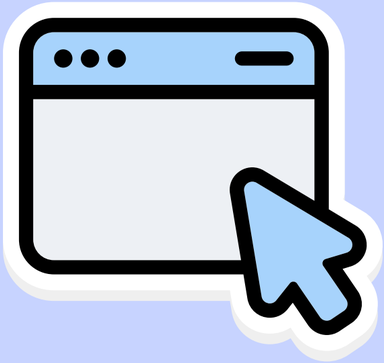
Generación de Respuestas (Handlers)

Rutas soportadas

- /lego/index.php → lista de figuras
- /lego/list.php → piezas por figura
- /log → bitácora del servidor
- /log/client → bitácora del cliente

Tecnologías usadas

- HTML → estructura de la página
- CSS → estilos visuales (colores, tablas, diseño)
- HTTP → protocolo de comunicación



Construcción de la respuesta

- Construcción de la respuesta
- buildHttpResponse()arma:
- Status (200 OK, 404, etc.)
- Headers (Content-Type, Content-Length)
- Body (HTML generado)

HTTPS / SSL

HTTP + capa de seguridad (SSL/TLS)

- Usa puerto 443
- Protege la comunicación (cifrado)
- Se utiliza SSLSocket en lugar de Socket

Proceso:

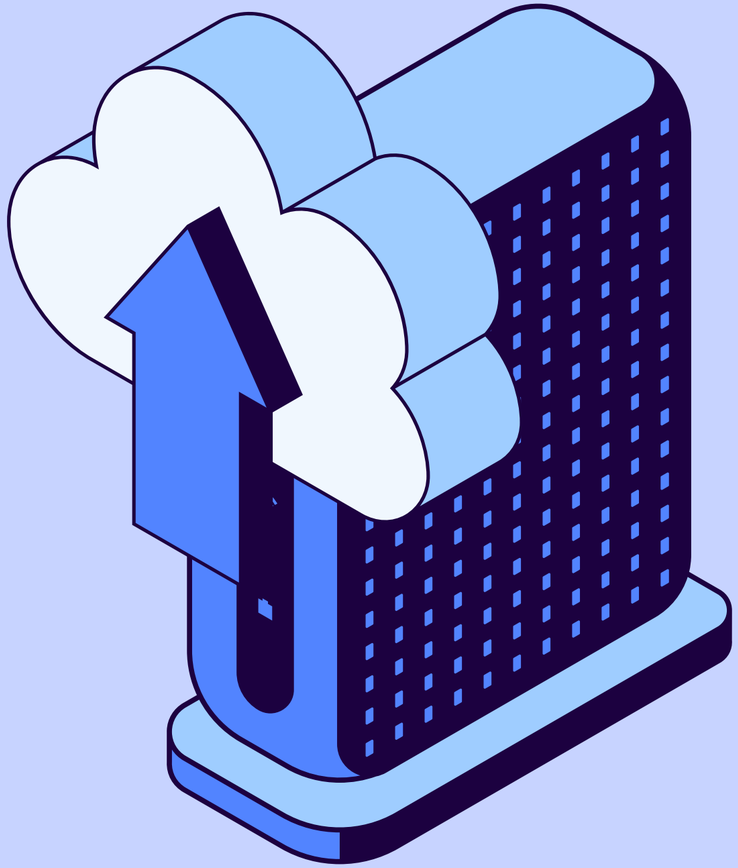
- Conexión inicial → TryToConnect()
- Asociación SSL → SSL_set_fd()
- Inicio de cifrado → SSL_connect()

Envío y recepción:

- SSL_write() → envía datos cifrados
- SSL_read() → recibe datos cifrados

Beneficios:

- Confidencialidad
- Integridad
- Autenticación



SERVIDOR + FILESYSTEM

- El servidor cuenta con su propio traductor de funciones del filesystem.
- Monta el disco.
- Usa las operaciones disponibles en el filesystem
- Mutex

Logs

- Mutex
- Acceso estatico
- Archivo y terminal
- Diferente para cliente y servidor

```
RVIDOR] [24/04/2026 11:31] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:31] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:32] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:32] [ERROR] Error procesando solicitud: Socket::Read( void *, size_t ): Co
SERVIDOR] [24/04/2026 11:32] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURE_NOT_FOUND figura=imperial_destroyee segmento=2
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURE_FOUND figura=imperial_destroyer segmento=2
SERVIDOR] [24/04/2026 11:33] [ERROR] Error procesando solicitud: Socket::Read( void *, size_t ): Co
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURE_FOUND figura=dragon segmento=1
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:33] [RESPUESTA] FIGURE_FOUND figura=dragon segmento=2
SERVIDOR] [24/04/2026 11:35] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:35] [RESPUESTA] FIGURE_FOUND figura=imperial_destroyer segmento=2
SERVIDOR] [24/04/2026 11:37] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:37] [RESPUESTA] FIGURES_LIST
SERVIDOR] [24/04/2026 11:38] [RESPUESTA] FIGURE_FOUND figura=imperial_destroyer segmento=1
SERVER] [26/04/2026 23:00] [RESPONSE] FIGURE_FOUND figure=fish segment=2
SERVER] [26/04/2026 23:00] [RESPONSE] FIGURES_LIST
SERVER] [26/04/2026 23:00] [RESPONSE] FIGURE_FOUND figure=death_star_ii segment=1
SERVER] [26/04/2026 23:00] [RESPONSE] FIGURE_FOUND figure=death_star_ii segment=2
SERVER] [26/04/2026 23:01] [RESPONSE] FIGURE_FOUND figure=death_star_ii segment=1
SERVER] [26/04/2026 23:01] [RESPONSE] FIGURE_FOUND figure=death_star_ii segment=2
SERVIDOR] [26/04/2026 23:02] [ERROR] Error procesando solicitud: HTTP method not supported
SERVER] [27/04/2026 21:32] [RESPONSE] FIGURES_LIST
ERVER] [27/04/2026 21:42] [RESPONSE] FIGURES_LIST
```

Cliente

- Es el encargado de conectarse al servidor, es exactamente el mismo de la etapa pasada.

Casos **de prueba** (servidor)

Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Servidor inicia correctamente	Ejecutar make run	Monta el filesystem y está listo para escuchar	Manejado correctamente
Ruta /lego/index.php	GET /lego/index.php	Devuelve HTML con el listado de figuras disponibles	Manejado correctamente
Ruta /lego/list.php válida	GET /lego/list.php?figure=horse&part=1	Devuelve tabla HTML con piezas de esa figura y mitad	Manejado correctamente
Figura inexistente	GET /lego/list.php?figure=noexiste&part=1	No muestra piezas inválidas y no se cae el servidor	Manejado correctamente
Mitad inexistente	GET /lego/list.php?figure=horse&part=9	No muestra piezas inválidas y no se cae el servidor	Manejado correctamente
Ruta inexistente	GET /algo	Devuelve 404 Not Found	Manejado correctamente
Request mal formado	Enviar texto incompleto	El servidor no se cae y responde error controlado	Manejado correctamente
Varias conexiones seguidas	Hacer varias consultas con curl	Todas reciben respuesta correcta	Manejado correctamente

Casos **de prueba** (servidor)

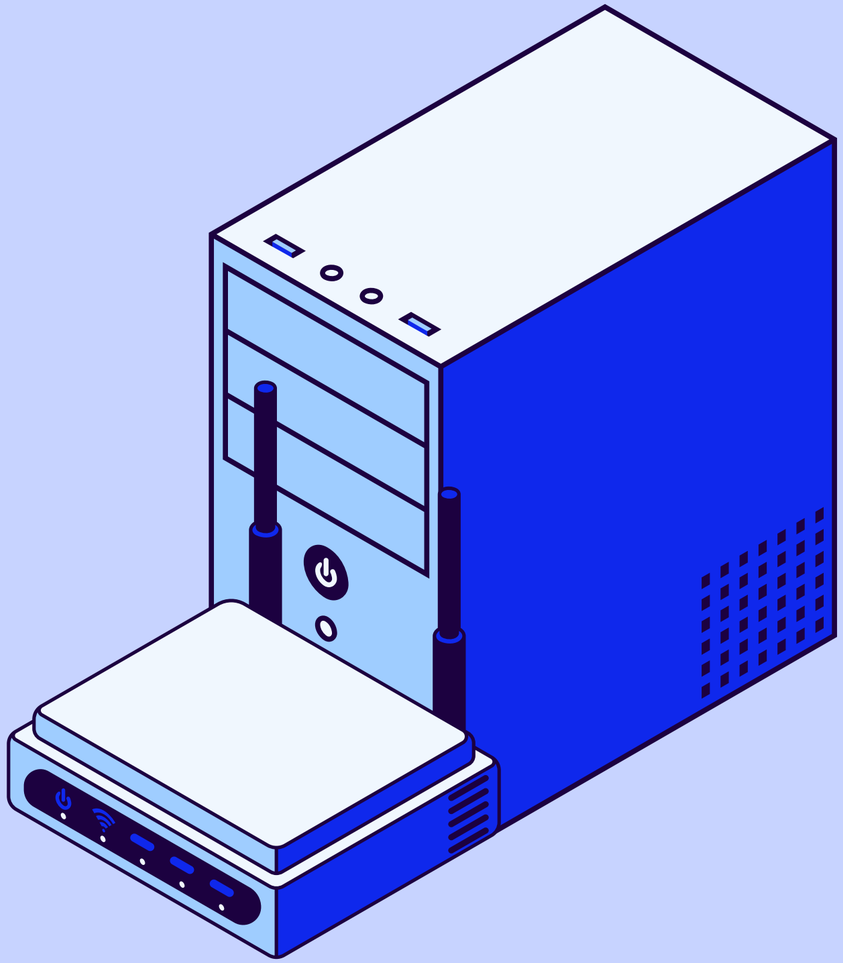
Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Concurrencia con hilos	Abrir varios clientes al mismo tiempo	El servidor atiende varias solicitudes sin bloquearse	Manejado correctamente
Bitácora registra listado	Consultar /lego/index.php	Se agrega evento FIGURES_LIST en bitacora.log	Manejado correctamente
Bitácora registra figura encontrada	Consultar figura válida	Se agrega FIGURE_FOUND figura=... segmento=...	Manejado correctamente
Bitácora registra figura no encontrada	Consultar figura inválida	Se agrega FIGURE_NOT_FOUND	Manejado correctamente
Bitácora en línea	Entrar a /bitacora	Se muestra el contenido de bitacora.log en HTML	Manejado correctamente
Bitácora vacía/no existente	Borrar bitacora.log y consultar /bitacora	Muestra mensaje de bitácora no existe o no pudo ser abierta	Manejado correctamente
Servidor sigue activo tras error	Enviar request inválido y luego uno válido	El servidor sigue respondiendo después del error	Manejado correctamente

Casos de prueba (FileSystem)

Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Formatear filesystem	Ejecutar ./fs_cli format 100	Se crea lego_data.bin, se inicializa metadata, bitmap, inodos y directorio root	Manejado correctamente
Montar filesystem	Ejecutar cualquier comando después de formatear, por ejemplo ./fs_cli ls	El filesystem se monta correctamente y muestra el bloque del inode root	Manejado correctamente
Listar filesystem vacío	Ejecutar ./fs_cli ls después de formatear	Muestra el directorio vacío sin errores	Manejado correctamente
Guardar archivo en filesystem	Ejecutar ./fs_cli put horse.txt horse	Se crea un inode, se guarda el contenido y confirma el almacenamiento	Manejado correctamente
Listar archivos guardados	Ejecutar ./fs_cli ls después de put	Muestra el nombre del archivo y su inode	Manejado correctamente
Leer archivo existente	Ejecutar ./fs_cli cat horse	Muestra el contenido almacenado en el filesystem	Manejado correctamente
Eliminar archivo existente	Ejecutar ./fs_cli rm horse	Elimina el archivo y libera bloques e inode	Manejado correctamente
Leer archivo eliminado	Ejecutar ./fs_cli cat horse después de borrarlo	Muestra Error: archivo no encontrado	Manejado correctamente

Casos **de prueba** (FileSystem)

Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Archivo duplicado	Ejecutar ./fs_cli put horse.txt horse dos veces	Rechaza la segunda creación con Error: archivo ya existe	Manejado correctamente
Disco no existente	Ejecutar ./fs_cli ls sin haber formateado	Muestra error indicando que no se pudo abrir lego_data.bin	Manejado correctamente
Comando desconocido	Ejecutar ./fs_cli abc	Muestra mensaje de comando desconocido y uso correcto	Manejado correctamente



Lab 3-5
Packet tracer

Etapa 2 - Equipo 3

Muchas **gracias**